

Code:

```
import java.util.Scanner;

public class BankersAlgorithm {
    private int processes, resources;
    private int[][] max, alloc, need;
    private int[] available;

    public BankersAlgorithm(int processes, int resources) {
        this.processes = processes;
        this.resources = resources;
        max = new int[processes][resources];
        alloc = new int[processes][resources];
        need = new int[processes][resources];
        available = new int[resources];
    }

    public void input(Scanner scanner) {
        System.out.println("Enter the available resources:");
        for (int i = 0; i < resources; i++) {
            System.out.print("Resource R" + i + ": ");
            available[i] = scanner.nextInt();
        }

        System.out.println("Enter the Max resources for each process:");
        for (int i = 0; i < processes; i++) {
            System.out.print("P" + i + ": ");
            for (int j = 0; j < resources; j++) {
                max[i][j] = scanner.nextInt();
            }
        }

        System.out.println("Enter the Allocated resources for each process:");
        for (int i = 0; i < processes; i++) {
            System.out.print("P" + i + ": ");
            for (int j = 0; j < resources; j++) {
                alloc[i][j] = scanner.nextInt();
                need[i][j] = max[i][j] - alloc[i][j];
            }
        }
    }
}
```

```

        available[j] -= alloc[i][j];
    }
}

}

public boolean isSafeState() {
    boolean[] finished = new boolean[processes];
    int[] work = available.clone();
    int count = 0;

    while (count < processes) {
        boolean found = false;
        for (int i = 0; i < processes; i++) {
            if (!finished[i]) {
                boolean canExecute = true;
                for (int j = 0; j < resources; j++) {
                    if (need[i][j] > work[j]) {
                        canExecute = false;
                        break;
                    }
                }
                if (canExecute) {
                    for (int j = 0; j < resources; j++) {
                        work[j] += alloc[i][j];
                    }
                    finished[i] = true;
                    found = true;
                    count++;
                }
            }
        }
        if (!found)
            break;
    }

    for (boolean f : finished) {
        if (!f)
            return false;
    }
    return true;
}

```

```

    }

    public void displayNeedMatrix() {
        System.out.println("Need Matrix:");
        for (int i = 0; i < processes; i++) {
            System.out.print("P" + i + ": ");
            for (int j = 0; j < resources; j++) {
                System.out.print(need[i][j] + " ");
            }
            System.out.println();
        }
    }

    public void displayAvailableResources() {
        System.out.print("Resources Remaining: ");
        for (int i = 0; i < resources; i++) {
            System.out.print(available[i] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of processes: ");
        int processes = scanner.nextInt();
        System.out.print("Enter the number of resources: ");
        int resources = scanner.nextInt();

        BankersAlgorithm bankers = new BankersAlgorithm(processes,
resources);
        bankers.input(scanner);

        bankers.displayNeedMatrix();
        bankers.displayAvailableResources();

        if (bankers.isSafeState()) {
            System.out.println("System is in a SAFE state.");
        } else {
            System.out.println("System is UNSAFE. Deadlock possible!");
        }
    }
}

```

```
}  
}  
}
```

Output:

```
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP6> javac BankersAlgorithm.java  
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP6> java BankersAlgorithm  
Enter the number of processes: 5  
Enter the number of resources: 3  
Enter the available resources:  
Resource R0: 10  
Resource R1: 5  
Resource R2: 7  
Enter the Max resources for each process:  
P0: 7 5 3  
P1: 3 2 2  
P2: 9 0 2  
P3: 2 2 2  
P4: 4 3 3  
Enter the Allocated resources for each process:  
P0: 0 1 0  
P1: 2 0 0  
P2: 3 0 2  
P3: 2 1 1  
P4: 0 0 2  
Need Matrix:  
P0: 7 4 3  
P1: 1 2 2  
P2: 6 0 0  
P3: 0 1 1  
P4: 4 3 1  
Resources Remaining: 3 3 2  
System is in a SAFE state.  
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP6>
```